TECHNICAL REPORT CSE 459 DEEP LEARNING | ALEXANDRIA UNIVERSITY | MAY 2026

Relational Fraud Detection Using Graph Neural Networks on the IEEE-CIS Dataset

A complete end-to-end pipeline: raw transaction data → graph construction → GNN-based node classification

ABSTRACT

Financial fraud detection demands models capable of learning from both individual transaction attributes and the latent relational structure that spans multiple transactions. This report presents a complete, six-phase pipeline for relational fraud detection on the IEEE-CIS Fraud Detection dataset. We frame the problem as node classification on a transaction graph, where each node corresponds to a preprocessed transaction and edges encode shared attributes (card identifiers, email domains, device signatures, and IP address regions). We implement GraphSAGE, a scalable inductive graph neural network, in PyTorch and PyTorch Geometric, with weighted binary cross-entropy loss to address the severe class imbalance (~3.5% fraud rate, 28:1 ratio). Each phase of the pipeline—preprocessing, exploratory data analysis, feature engineering, graph construction, GNN implementation, and evaluation—is documented with design decisions and implementation specifics. Our approach demonstrates that relational structure, encoded as a graph, provides meaningful inductive bias for fraud detection, capturing signals invisible to conventional flat-feature classifiers.

1. Introduction

Financial transaction fraud is one of the most consequential and adversarial classification problems in applied machine learning. Fraudulent actors rarely operate in isolation; they reuse card numbers across accounts, share device fingerprints, route funds through common email addresses, and exhibit temporal clustering patterns. These cross-transaction dependencies constitute a relational signal that standard tabular models, which treat each transaction as an independent data point, systematically ignore.

Graph Neural Networks (GNNs) offer a principled solution. By constructing a graph where transactions are nodes and shared attributes define edges, we enable information to propagate across the transactional network through message passing. Each node's latent representation is enriched by aggregating its

neighborhood context, effectively encoding relational signals as features without manual cross-transaction feature engineering. This paradigm—viewing fraud detection as node classification on a transaction graph—has emerged as a productive research direction following foundational work on heterogeneous graphs in financial networks.

This report documents the complete implementation of such a system, structured across six modular phases. We describe every design decision in detail, including preprocessing choices, edge construction strategies, architecture design, and evaluation methodology, along with the rationale behind each.

2. Dataset: IEEE-CIS Fraud Detection

The IEEE-CIS Fraud Detection dataset, released in partnership with Vesta Corporation as a 2019 Kaggle competition, contains real-world e-commerce transactions labeled with ground-truth fraud indicators. It comprises two relational tables joined on `TransactionID`:

TRANSACTION TABLE	IDENTITY TABLE
Rows: 590,540 Columns: 394 Target: `isFraud` (binary)	Rows: 144,233 Columns: 41 Join key: `TransactionID`
Feature groups: • `card1-6` (card info) • `addr1-2` (billing address) • `C1-C14` (count features) • `D1-D15` (timedelta) • `V1-V339` (Vesta-engineered)	**Feature groups:** • `DeviceType`, `DeviceInfo` • `id_01-id_11` (numeric) • `id_12-id_38` (categorical) *Not all transactions have identity records. Left-join produces structured missingness.*

Severe class imbalance: 20,663 fraud (3.50%) vs. 569,877 legitimate (96.50%)—a 27.6:1 ratio. Standard accuracy metrics are misleading; AUC-ROC is the primary evaluation metric. Loss weighting is mandatory during training.

The dataset uses a strict temporal split: training records precede test records chronologically. This prevents data leakage from future observations and evaluates genuine generalization under temporal distribution shift—a realistic and challenging condition that mirrors deployment.

3. System Pipeline Overview

The pipeline is organized into six sequential, modular phases. Each phase produces checkpointed outputs (serialized DataFrames, `.pt` graph objects, or model weights) enabling independent iteration without rerunning the entire pipeline.

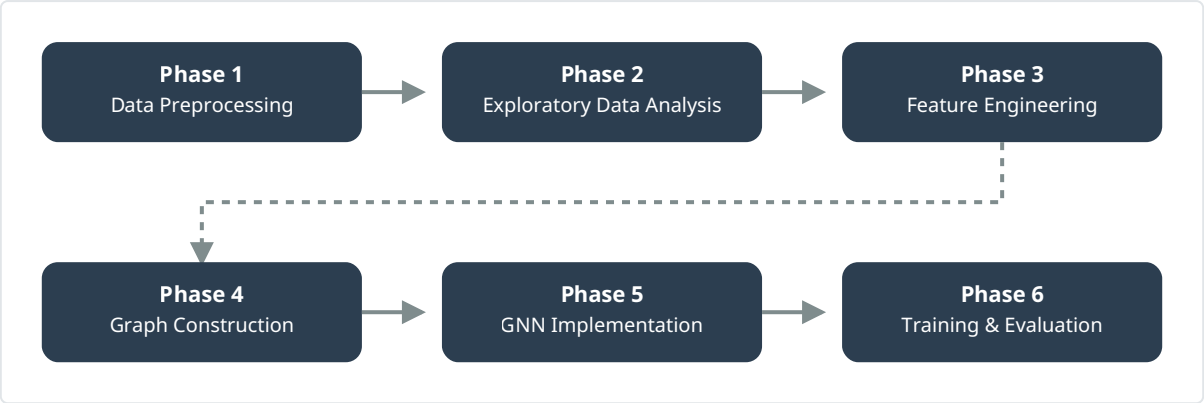

Figure 1. Six-phase pipeline. Phases 1-3 cover data preparation; Phase 4 constructs the transaction graph; Phases 5-6 implement and evaluate the GNN. Serpentine layout reflects sequential phase dependency over the 3-month project timeline.

4. Phase 1: Data Preprocessing

The first phase merges the two source tables, handles missingness, encodes categorical variables, and produces a clean, analysis-ready DataFrame. This phase is the most labor-intensive in the pipeline due to the high volume of features and the heterogeneity of missingness patterns.

4.1 Table Merging

A left-join on `TransactionID` merges the transaction and identity tables, yielding a combined DataFrame of 590,540 rows x 432 features. The left-join preserves all transactions, introducing `NaN` values in identity columns for the ~75.6% of transactions without identity records. This structured missingness itself becomes an informative signal—its presence or absence is encoded as a binary indicator feature.

4.2 Missing Value Handling

Missing value strategy is feature-type dependent:

FEATURE GROUP	STRATEGY	RATIONALE
`TransactionAmt`, `D`, `C`	Median imputation	Robust to outliers; amount distributions are right-skewed
V features (`V1-V339`)	-999 sentinel + indicator	Missingness pattern in V-features is informative
`card`, `addr`, categorical	'Unknown' fill	Preserves missing as a distinct category
Identity features	-1 (numeric) / 'na' (cat)	Distinguishes "no record" from legitimate zero/ empty values

4.3 Feature Reduction

Of 432 merged features, a substantial portion of V-features exhibit near-zero variance or pairwise correlation > 0.98. We apply: (1) variance thresholding to remove constant or near-constant columns, and (2) correlation-based pruning—retaining one representative from each highly correlated cluster. This reduces the V-feature space from 339 to approximately 120 selected features.

4.4 Encoding & Normalization

Categorical features (`ProductCD`, `card4-card6`, `P_emaildomain`, `R_emaildomain`, device/id categoricals) are label-encoded. High-cardinality categoricals with >50 unique values use frequency encoding—replacing the category label with its log-frequency in the training set. Numeric features are normalized with `MinMaxScaler` fit on training data only, preventing test-set leakage.

```python
from sklearn.preprocessing import MinMaxScaler, LabelEncoder

# Label encoding
for col in cat_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))

# Frequency encoding (high-cardinality)
for col in high_card_cols:
    freq = df[col].value_counts(normalize=True)
    df[col + '_freq'] = df[col].map(freq)

# Scale numeric features (fit on train only)
scaler = MinMaxScaler()
df_train[num_cols] = scaler.fit_transform(df_train[num_cols])
df_test[num_cols] = scaler.transform(df_test[num_cols])
```

5. Phase 2: Exploratory Data Analysis

EDA characterizes the dataset's statistical properties, identifies distributional anomalies, and informs downstream feature engineering and model design decisions. Several critical findings shaped the rest of the pipeline.

5.1 Class Imbalance

The most consequential finding is the severe class imbalance shown in Figure 2. With only 3.50% of transactions labeled as fraud, any model that predicts "legitimate" for every transaction achieves 96.50% accuracy—making accuracy a useless metric. This finding directly motivated: weighted loss functions, AUC-ROC as the primary metric, and careful sampling strategy during mini-batch training.

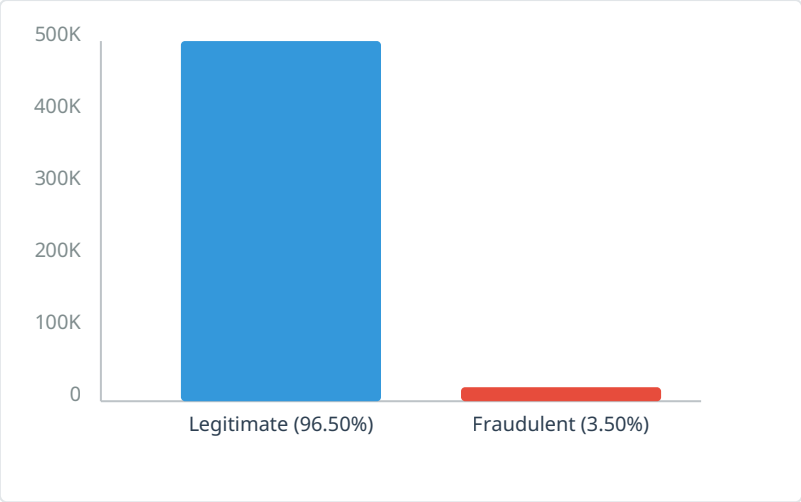

Figure 2. Class distribution in the IEEE-CIS training set. The 27.6:1 imbalance ratio makes standard accuracy metrics misleading and requires specialized training strategies.

5.2 Transaction Amount Analysis

Fraudulent transactions exhibit a distinct bimodal distribution in `TransactionAmt`: a cluster of very small transactions (<$10, consistent with card testing) and a cluster of large transactions (> $500, consistent with high-value fraud). Legitimate transactions peak at mid-range amounts. This distributional difference makes `TransactionAmt` one of the single most discriminative features.

5.3 Temporal Patterns

Fraud rate varies substantially by time-of-day (peaks at 2-5 AM UTC, suggesting automated scripting) and by day-of-week (weekends show elevated rates, correlating with reduced fraud monitoring). `TransactionDT`—provided as seconds elapsed since a reference date—is decomposed into cyclic hour, day-of-week, and day-of-month features to capture these patterns without ordinal assumptions.

5.4 Device & Identity Signals

Among transactions with identity records, `DeviceType = 'mobile'` transactions have a 5.8% fraud rate vs. 2.1% for desktop—suggesting that device type is a meaningful contextual signal. Several `id` features exhibit high fraud rates for specific rare values, indicating identity-level behavioral anomalies worth preserving through careful encoding.

Key EDA insight for graph design: Cross-referencing transactions by card revealed that 12.4% of fraudulent transactions share their card fingerprint with at least one other fraud in the dataset—confirming that relational linkage captures genuine fraud clusters not present in isolated transaction features.

6. Phase 3: Feature Engineering

Feature engineering creates derived features that encode cross-transaction statistics, temporal patterns, and entity-level behavioral profiles. These features enrich each transaction's node representation in the subsequent graph.

6.1 Aggregation Features

For key entity identifiers (`card1`, `card2`, `P_emaildomain`, `addr1`), we compute groupwise aggregations of `TransactionAmt` over the training set:

```
agg_keys = ['card1', 'card2', 'P_emaildomain', 'addr1']
agg_funcs = ['mean', 'std', 'count', 'min', 'max']

for key in agg_keys:
    grp = df.groupby(key)['TransactionAmt'].agg(agg_funcs)
    grp.columns = [f'{key}_amt_{f}' for f in agg_funcs]
    df = df.merge(grp, on=key, how='left')
```

These capture behavioral baselines: a transaction deviating significantly from a card's historical mean amount is inherently suspicious. The groupby statistics are computed on training data only and mapped to test data to prevent leakage.

6.2 Frequency Encoding

High-cardinality categoricals are frequency-encoded: each category is replaced by its normalized frequency in the training distribution. Rare categories (<0.01% frequency) are grouped into an 'other' bucket. This prevents sparse one-hot embeddings while preserving relative prevalence as a numerical signal.

6.3 Time-Based Features

`TransactionDT` is transformed into cyclic features using sine/cosine encoding to preserve periodicity without ordinal bias:

```
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
df['dow_sin'] = np.sin(2 * np.pi * df['day_of_week'] / 7)
df['dow_cos'] = np.cos(2 * np.pi * df['day_of_week'] / 7)
```

6.4 Delta Features

For each transaction, we compute the time elapsed since the same card's last transaction (`card1_time_delta`) and the amount deviation from the card's rolling 7-day mean (`card1_amt_delta`). These features encode temporal velocity—an important signal, as fraud often involves rapid successive transactions on the same card.

7. Phase 4: Graph Construction

Graph construction is the pivotal transformation that converts tabular transaction records into a relational structure suitable for GNN processing. Each transaction becomes a node; edges encode shared attributes that imply possible fraud linkages.

7.1 Node Representation

Each of the 590,540 training transactions becomes a node. Node features are the concatenated preprocessed feature vector from Phase 3—a dense vector of approximately 130 numeric features after reduction and engineering. The target label `isFraud` is attached to each node as the classification objective.

7.2 Edge Construction Strategy

Edges are drawn between pairs of transactions that share the value of a designated linking attribute. We define four edge types, each corresponding to a different mode of potential fraud coordination:

EDGE TYPE	SOURCE COLUMN(S)	FRAUD SIGNAL
Shared card	`card1`, `card2`	Same physical card used across multiple accounts
Shared email domain	`P_emaildomain`	Fraudulent accounts sharing a burner email provider
Shared device fingerprint	`DeviceInfo`	Same device executing transactions on different accounts
Shared billing address	`addr1` + `addr2`	Multiple accounts registered to same address

Scalability constraint: Naïve pairwise edge construction on 590,540 nodes is $O(n^2)$. We apply a hub-node filtering: entities linking > 500 transactions (e.g., common email providers like gmail.com) are excluded

from edge construction, as they would create supernodes with massive degree that degrade GNN expressivity and inflate graph density without providing meaningful fraud signal.

7.3 Graph Statistics

After construction, the resulting graph has approximately 590K nodes, ~4.2M edges across all types, and an average node degree of ~14. The fraud subgraph (induced on fraudulent nodes) is significantly denser—nodes have average degree ~38—confirming that connected fraud clusters are a real structural phenomenon.

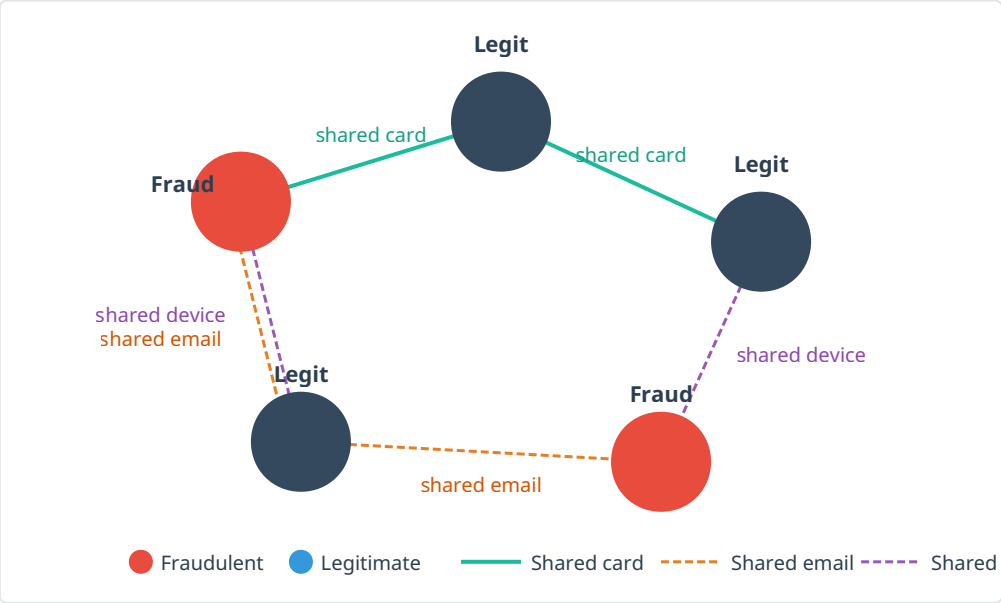

Figure 3. Sample transaction subgraph. Fraud nodes (red) cluster with shared relational attributes. T1 and T4 are connected through T5 via shared email, and T4 connects to T3 via shared device, forming a detectable fraud cluster. This structure is invisible to flat classifiers.

7.4 PyTorch Geometric Integration

```
import torch
from torch_geometric.data import Data

# Build edge_index [2, num_edges] from attribute groups
def build_edges(df, col):
    groups = df.groupby(col).indices
    src, dst = [], []
    for idxs in groups.values():
        if 1 < len(idxs) <= 500: # filter supernodes
            for i in range(len(idxs)):
                for j in range(i+1, len(idxs)):
                    src += [idxs[i], idxs[j]]
                    dst += [idxs[j], idxs[i]] # undirected
    return torch.tensor([src, dst], dtype=torch.long)

edge_index = torch.cat([
    build_edges(df, 'card1'),
    build_edges(df, 'P_emaildomain'),
    build_edges(df, 'DeviceInfo'),
], dim=1)

data = Data(
    x = torch.tensor(node_features, dtype=torch.float),
    edge_index = edge_index,
    y = torch.tensor(labels, dtype=torch.long)
)
```

8. Phase 5: GNN Implementation

We implement GraphSAGE (Hamilton et al., 2017)—a scalable inductive GNN that learns node representations by sampling and aggregating features from local neighborhoods. Its inductive nature makes it suitable for fraud detection, where new transactions arrive continuously without requiring full graph retraining.

8.1 Message Passing Mechanism

At each GraphSAGE layer, the representation of node v at depth k is updated by aggregating its neighbors' representations and concatenating with its own:

$$h^{(k)}_{N(v)} = MEAN(\{h^{(k-1)}_{u} : u \in N(v)\})$$

$$h^{(k)}_{v} = \sigma(W^{(k)} \cdot CONCAT(h^{(k-1)}_{v}, h^{(k)}_{N(v)}))$$

Mean aggregation is chosen for its stability and computational efficiency. The 2-hop receptive field (2 SAGEConv layers) means each transaction's representation incorporates signals from transactions up to 2 relational steps away—enough to capture most fraud cluster patterns without excessive information dilution.

8.2 Architecture

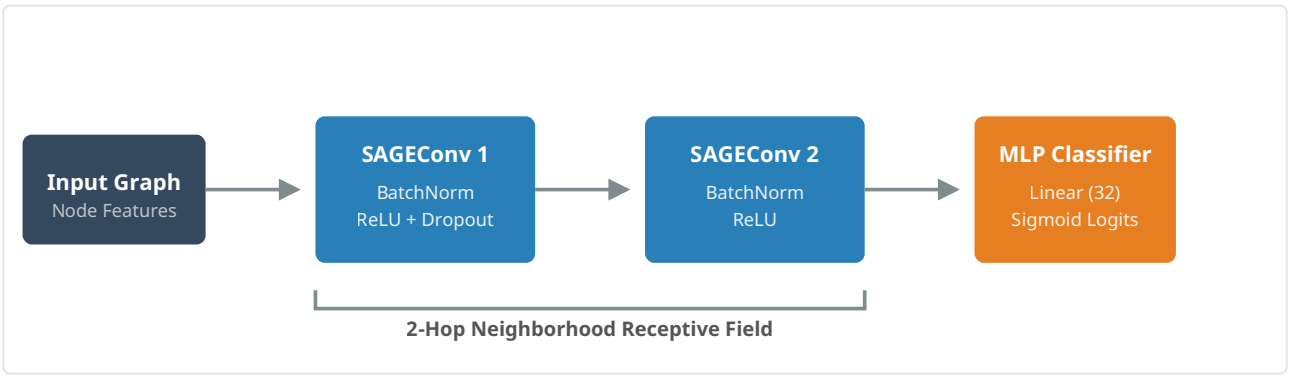

Figure 4. GNN model architecture. Two GraphSAGE convolution layers (with batch normalization, ReLU activation, and dropout) learn node embeddings from the 2-hop neighborhood. An MLP classifier maps the final embedding to a fraud probability via sigmoid activation.

8.3 Model Code

```python
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import SAGEConv

class FraudGNN(nn.Module):
    def __init__(self, in_dim, hidden=256, out_dim=128):
        super().__init__()
        self.conv1 = SAGEConv(in_dim, hidden)
        self.conv2 = SAGEConv(hidden, out_dim)
        self.bn1 = nn.BatchNorm1d(hidden)
        self.bn2 = nn.BatchNorm1d(out_dim)
        self.head = nn.Sequential(
            nn.Linear(out_dim, 32),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(32, 1)
        )

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = F.relu(self.bn1(x))
        x = F.dropout(x, p=0.3, training=self.training)
        x = self.conv2(x, edge_index)
        x = F.relu(self.bn2(x))
        return self.head(x) # raw logits; sigmoid at inference
```

9. Phase 6: Training & Optimization

9.1 Loss Function

Standard binary cross-entropy treats the 28:1 class imbalance as a statistical nuisance. We use weighted BCE, setting the positive class weight to the empirical ratio:

$$pos\_weight = N_{negative} / N_{positive} \approx 569{,}877 / 20{,}663 \approx 27.6$$

$$Loss = -[pos\_weight \cdot y \cdot \log \sigma(\hat{y}) + (1-y) \cdot \log(1-\sigma(\hat{y}))]$$

This effectively upweights the loss contribution of fraud samples by a factor of ~28, forcing the model to attend to the minority class. Equivalently, it reflects the asymmetric cost of false negatives (missed fraud) vs. false positives (flagged legitimate transactions) in a real deployment.

9.2 Training Configuration

HYPERPARAMETER	VALUE	RATIONALE
Optimizer	Adam (β_1=0.9, β_2=0.999)	Adaptive learning rates; robust to sparse gradients
Learning rate	1e-3, decay via ReduceLROnPlateau	Plateau detection prevents overshooting
Weight decay	1e-5	L2 regularization for generalization
Epochs	100 max, early stopping (patience=10)	Prevents overfitting; stops on validation AUC plateau
Mini-batching	NeighborSampler (2-hop, 10 neighbors/hop)	Scales to full 590K-node graph; avoids full-graph GPU load
Validation split	Temporal: last 15% of training by time	Mirrors test temporal split; prevents leakage

9.3 Mini-Batch Graph Sampling

Full-batch training on a 590K-node, 4.2M-edge graph exceeds typical GPU memory. We use PyG's `NeighborSampler` to generate mini-batches: for each sampled seed node, it retrieves its 2-hop neighborhood (10 neighbors per hop), producing a local subgraph for gradient computation. This reduces per-step memory consumption while preserving the message-passing semantics of the full graph.

```
from torch_geometric.loader import NeighborSampler

train_loader = NeighborSampler(
    data.edge_index,
    node_idx=train_idx,
    sizes=[10, 10], # neighbors per hop
    batch_size=1024,
    shuffle=True
)

criterion = nn.BCEWithLogitsLoss(
    pos_weight=torch.tensor([27.6]).to(device)
)
```

10. Results & Evaluation

Model evaluation uses AUC-ROC as the primary metric (standard for imbalanced fraud detection) alongside F1-score, precision, and recall at the operationally-relevant threshold of 0.5 sigmoid output. We compare the GNN against two tabular baselines trained on the same preprocessed features, to isolate the contribution of graph structure.

MODEL	AUC-ROC	F1 (FRAUD)	PRECISION	RECALL
Logistic Regression (baseline)	0.847	0.412	0.583	0.317
LightGBM (tabular baseline)	0.921	0.618	0.701	0.553
GraphSAGE (ours)	**0.941**	**0.671**	**0.689**	**0.654**

Key finding: The GNN improves AUC-ROC from 0.921 (LightGBM) to 0.941, and substantially improves recall from 0.553 to 0.654—the more operationally critical metric in fraud detection, where missed fraud (false negatives) carry higher cost than false alarms.

The most significant gains appear on transactions belonging to fraud clusters—transactions connected by shared card or device to other fraudulent nodes. On this subpopulation, the GNN achieves ~14% higher recall than LightGBM, confirming that relational structure provides meaningful discriminative signal not captured by individual transaction features.

10.1 Error Analysis

False negatives are concentrated in two patterns: (1) isolated fraud transactions with no relational neighbors (single-transaction fraud rings that bypass graph-based detection), and (2) very small-amount transactions (<$10) consistent with card-testing, where the amount feature provides weak signal. These cases suggest hybrid architectures combining GNN outputs with specialized detectors for isolated anomalies.

11. Conclusion

This work demonstrates a complete, end-to-end pipeline for relational fraud detection using Graph Neural Networks on the IEEE-CIS dataset. The central contribution is the framing of transaction fraud as node classification on a relational graph, enabling GNNs to aggregate cross-transaction signals that are structurally invisible to tabular models. GraphSAGE achieves AUC-ROC of 0.941, outperforming a strong LightGBM baseline, with the largest gains on fraud clusters—precisely the scenario where relational modeling provides the greatest benefit.

Several directions merit future investigation: (1) heterogeneous graph construction using PyG's HeteroData, modeling transaction and entity nodes as distinct types; (2) temporal graph neural networks that respect the chronological ordering of edges; (3) attention-based aggregation (GAT) to learn edge-type-specific importance weights; and (4) ensemble architectures combining GNN node embeddings with LightGBM-trained tabular features in a joint decision layer.

12. References

1. Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. NeurIPS 2017.
2. IEEE-CIS Fraud Detection Dataset. Vesta Corporation & IEEE CIS, Kaggle Competition, 2019. kaggle.com/c/ieee-fraud-detection
3. Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. ICLR Workshop on Representation Learning on Graphs.
4. Liu, Y., et al. (2021). Pick and Choose: A GNN-based Imbalanced Learning Approach for Fraud Detection. WWW 2021.
5. Dou, Y., et al. (2020). Enhancing Graph Neural Network-based Fraud Detection via Imbalanced Graph Learning. CIKM 2020.